Member-only story

Word Embeddings | Word2vec | Machine Learning | Linear Algebra | Artificial Intelligence

Why King – Man + Woman = Queen: What a Word Embedding Actually Is

Word2vec, the 2013 method everyone called a neural network, was factorizing a word-pair table linguists had used since 1990.



Mathcube Following 11 min read · 10 hours ago



Photo by Brett Jordan on Unsplash

In 2013 a group of researchers at Google described a piece of arithmetic that has since opened almost every introduction to machine learning. Take the list of numbers their program, word2vec, had assigned to the word *king*, subtract the list for *man*, add the list for *woman*, and ask which word's list lies closest to the result. The program answers *queen*. The word vectors Google released that year had been trained on roughly a hundred billion words of news text, with no dictionary, no grammar and no information about monarchy or sex, only a record of which words turn up near which others.

The demonstration raises two questions. How can subtracting one word from another mean anything at all? And did the arithmetic really say *queen*? Popular accounts tend to answer only the first, and loosely. The first has a precise answer, and it runs through a table of word statistics that linguists were computing more than a decade before the first neural language

models. The second has an answer too, though it flatters the demonstration less.

The company a word keeps

In 1954 the linguist Zellig Harris argued that words occurring in similar contexts tend to have similar meanings, and in 1957 J. R. Firth gave the idea the line it is still quoted by: “You shall know a word by the company it keeps.”

Neither man had a way to measure company at scale, but the measurement is easy to describe. Take a large body of text and a vocabulary of, say, 100,000 words. Build a grid with one row for every word and one column for every word, and in each cell count how often the column word appears within a few positions of the row word. The row for *coffee* will have large entries under *cup*, *drink* and *morning*; the row for *tea* will have large entries in many of the same columns. Each row is now a list of 100,000 numbers, which is to say a point in a space with one axis per context word. A *word embedding* is a placement of words as points in a space, chosen so that the geometry of the points records which words keep company with which; nearby points are words that turn up in similar surroundings.

Straight-line distance between two such points makes a poor measure of similarity, because a common word such as *house* has larger counts in every column than a rare word such as *cottage*, and its point sits much farther from the origin even where the two words keep identical company. The standard remedy is to ignore length and measure only direction, using the cosine of the angle between two rows:

$$\cos\theta = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|}$$

Here **u** and **v** are the two rows, the numerator is their dot product (multiply matching entries and add them up), and the denominator divides out both lengths. For rows of counts, which contain no negative entries, the cosine runs from 0 to 1. For the learned vectors later in the piece, whose entries can be negative, it can fall as low as −1. A value near 1 means the two words

Open in app ↗

Medium Search Play Games Write

Raw counts cause two further problems for anyone building such a table from real text. The table is enormous and almost entirely zeros, since most pairs of words never appear near each other in any corpus. And the columns for *the*, *of* and *and* dominate every row, because those words sit next to everything. Both problems call for replacing each count with a measure of how surprising the pair's frequency is.

What word2vec actually did

The work behind the famous equation appeared in two 2013 papers by Tomáš Mikolov and colleagues at Google: the first proposed two simple models for learning word vectors, one of them called skip-gram, together with a test set of analogies, and the second added negative sampling, a simpler and faster training trick that also gave better vectors. The software released with the papers was called word2vec.

In skip-gram with negative sampling, every word in the vocabulary receives two short vectors of, typically, 300 numbers each, one used when the word is

the focus and one, its context vector, used when it appears as somebody else's neighbour. A window slides along the text. For every genuine pair it finds, a word *w* with a neighbour *c*, the training nudges the two vectors to point more nearly the same way. Then it draws *k* words at random from the vocabulary, typically around five, and up to about twenty, treats them as fake neighbours, and nudges their context vectors to point away from the focus word's vector. For each genuine pair, training pushes up a sum of two terms, one for the real neighbour and one for the fakes:

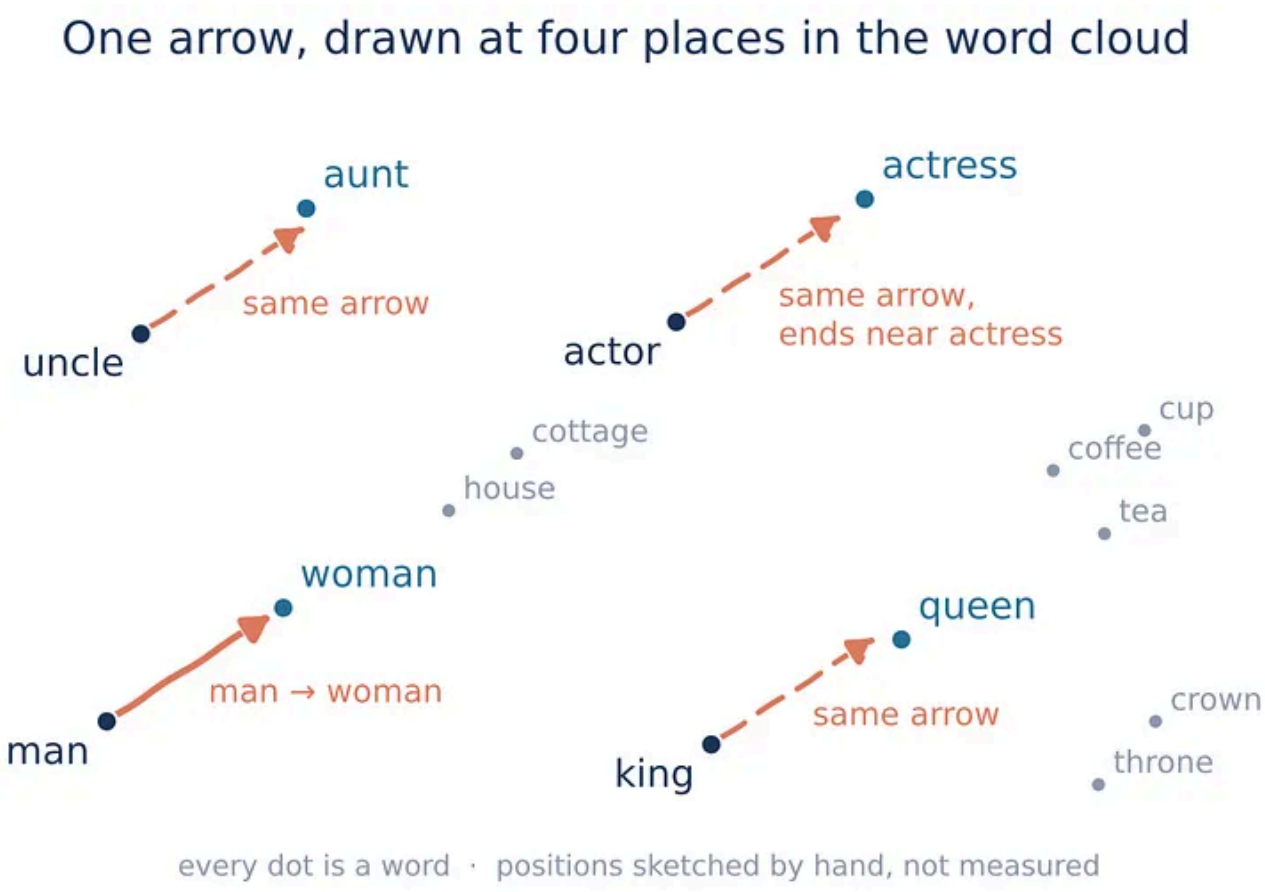
$$\log \sigma(\mathbf{w} \cdot \mathbf{c}) \; + \; k \, \mathbb{E}_{c'} \left[\log \sigma(-\mathbf{w} \cdot \mathbf{c}') \right]$$

where $\sigma(x) = 1/(1 + e^{-x})$ is the logistic function, which turns any number into a probability between 0 and 1. The first term is the model's log-probability that the pair (*w*, *c*) is genuine, and it grows as the dot product $\mathbf{w}\cdot\mathbf{c}$ grows. The second term averages over fake neighbours c' drawn at random, weighted by *k*, and it grows as $\mathbf{w}\cdot\mathbf{c}'$ becomes negative. Pushing the whole expression up teaches the model to tell genuine neighbours from noise. The program stores no co-occurrence counts, and the table from the previous section appears nowhere in the code.

Press coverage described the method as a neural network, which is generous, since there is no nonlinear hidden layer; Mikolov's first paper itself calls the models log-linear. Training leaves behind a matrix *W* with one row of 300 numbers per word, the focus vectors. The context vectors are usually discarded, although the argument in the next section needs both. The surviving table is the kind every deep-learning framework ships as its embedding layer (`torch.nn.Embedding` in PyTorch), and the first thing a language model does to each incoming token is look up its row.

The analogy test computes **king** − **man** + **woman** and then finds the vocabulary word whose vector has the highest cosine with the result. On the 300-dimensional vectors Google trained on its news corpus, which cover three million words and phrases, the winner is *queen*, at a cosine of about 0.71 (0.7118 in the popular gensim library).

The geometric reading is that the step from *man* to *woman* is a fixed direction in the space. Draw that arrow starting at *king* and it ends near *queen*; start it at *uncle* and it ends near *aunt*, at *actor* and it ends near *actress*. The same arrow, laid down at different places in a cloud of three million points, lands on the right word roughly two times in three on the standard test set.



The step from man to woman, copied unchanged to three other words; a 2-D sketch, not measured positions.

The matrix nobody prints

In 2014 Omer Levy and Yoav Goldberg, then at Bar-Ilan University, asked what negative sampling converges to. Their approach was to set the derivative of the objective to zero for each (word, context) pair separately, under the assumption that the vectors have enough dimensions to satisfy every pair at once. The answer involves a quantity introduced to computational linguistics by Kenneth Church and Patrick Hanks in a paper first presented in 1989 and published in 1990, the pointwise mutual information:

$$\mathrm{PMI}(w, c) = \log \frac{P(w, c)}{P(w)\,P(c)}$$

The numerator is the probability of finding *w* and *c* near each other in the text; the denominator is the probability the pair would have if the two words were scattered independently. PMI is the log of their ratio, so it is zero for a pair that meets exactly as often as chance predicts, positive for a pair that meets more often, and negative for one that avoids each other. It is the better cell entry that the first section asked for, since it discounts *the* and *of*automatically: they appear next to everything, so they appear next to any particular word about as often as chance predicts.

Levy and Goldberg's result is that at the optimum of the negative-sampling objective,

$$\mathbf{w} \cdot \mathbf{c} = \mathrm{PMI}(w, c) - \log k$$

The dot product of a word's vector with a context's vector equals the log of how much more often the pair occurs than chance, minus a constant that depends only on the number of fake neighbours. Stack all the word vectors as the rows of a matrix W and all the context vectors, the set training usually throws away, as the rows of a matrix C, and the statement becomes $W C^T \approx M$, where M is the big word-by-context table with every entry set to PMI minus log k. Writing one matrix as the product of two thinner ones is called a matrix factorization, and the singular value decomposition, or SVD, is the standard way to compute one. Skip-gram with negative sampling, read this way, is factorizing a shifted PMI matrix without ever building it.

The identity assumes vectors with enough dimensions to fit every entry of the matrix. With 300 numbers per word, stochastic gradient descent settles for an approximation that weights frequent pairs more heavily, so word2vec and an explicit factorization of the PMI table aim at the same matrix by different routes, and their vectors come out close but not identical. Levy and Goldberg ran an SVD on the shifted PMI matrix with its negative entries set to zero, since a pair that never co-occurs has a PMI of minus infinity and pairs rarer than chance carry little usable information. The result matched skip-gram on word-similarity benchmarks, while skip-gram stayed ahead on analogies. A 2015 follow-up showed that much of the wider gap between learned embeddings and count-based methods came from training choices the matrix methods had skipped, such as word2vec's smoothing of context frequencies, which raises each word's count to the power 0.75 before fake neighbours are drawn and helps consistently across tasks.

GloVe, published in 2014, builds the co-occurrence table on purpose and fits word vectors to the logarithms of its counts. Latent semantic analysis, published in 1990 like the PMI paper, used an SVD to factorize a different table, with a row per term and a column per document, filled with weighted counts.

One arrow, drawn everywhere

The factorization also explains the arrow. Suppose each word's vector encodes its row of PMI values, in the sense that its dot product with any context vector gives the PMI of that pair. Take the difference **king** − **man** and dot it with a context vector **c**. By the definition of PMI, each of the two dot products is log $P(w, c)$ − log $P(w)$ − log $P(c)$ − log k. Subtracting the one for *man* from the one for *king* removes the last two terms, which are the same for both words. The remainder is log [P(king, c)/P(king)] minus the same expression for *man*, and P(king, c)/P(king) is just $P(c \mid \text{king})$, the probability of finding c given that the word is *king*. The difference is a log-ratio of two conditional probabilities:

$$(\mathbf{king} - \mathbf{man}) \cdot \mathbf{c} \;\approx\; \log \frac{P(c \mid \text{king})}{P(c \mid \text{man})}$$

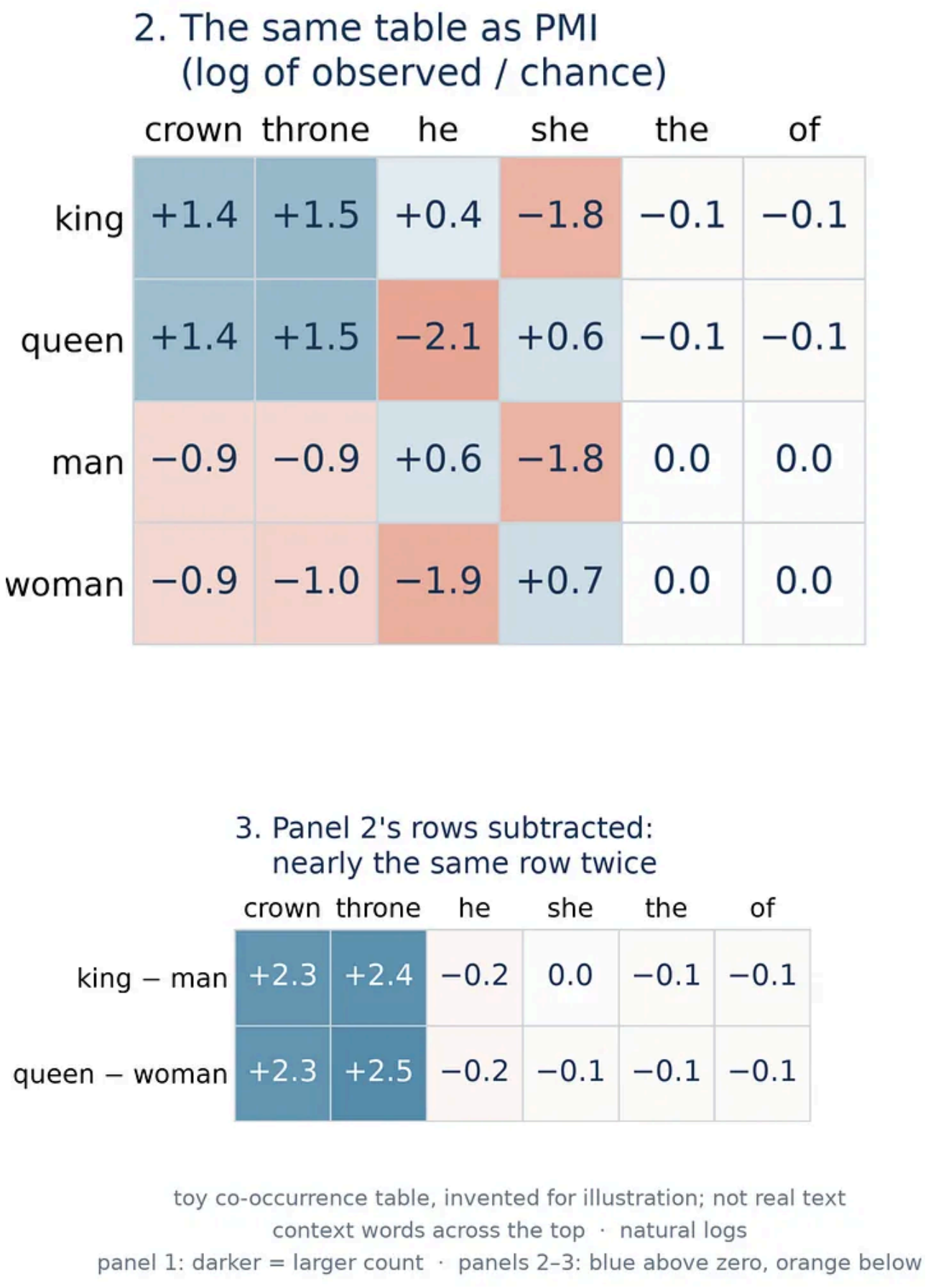
In words, the difference vector records, for every context word c, the log of how much more likely c is to appear near *king* than near *man*. For *crown*, *throne* and *reign* that ratio is large; for *he*, *his* and *himself* it is close to one, because both words attract them.

Nothing in the arithmetic guarantees that **queen** − **woman** has the same profile. The two arrows match to the extent that the corpus treats royalty and

gender as independent factors in a word's company. *Crown, throne* and *reign*depend on royalty and not on gender, so their ratios come out the same whether the pair is *king* and *man* or *queen* and *woman*. *He, his* and *himself*, like *she* and *her*, depend on gender and not on royalty, so they cancel within each pair. This is an empirical regularity of how people use words, roughly true in a large corpus. It fails whenever the two pairs differ in more than one respect at once. Where it holds, the two difference vectors give the same log-ratio for every context, they are the same vector as far as the geometry can tell, and the arrow from *man* to *king* coincides with the arrow from *woman* to *queen*. Rearranged, **king − man + woman ≈ queen**.

1. Counts: how often the column word appears near the row word

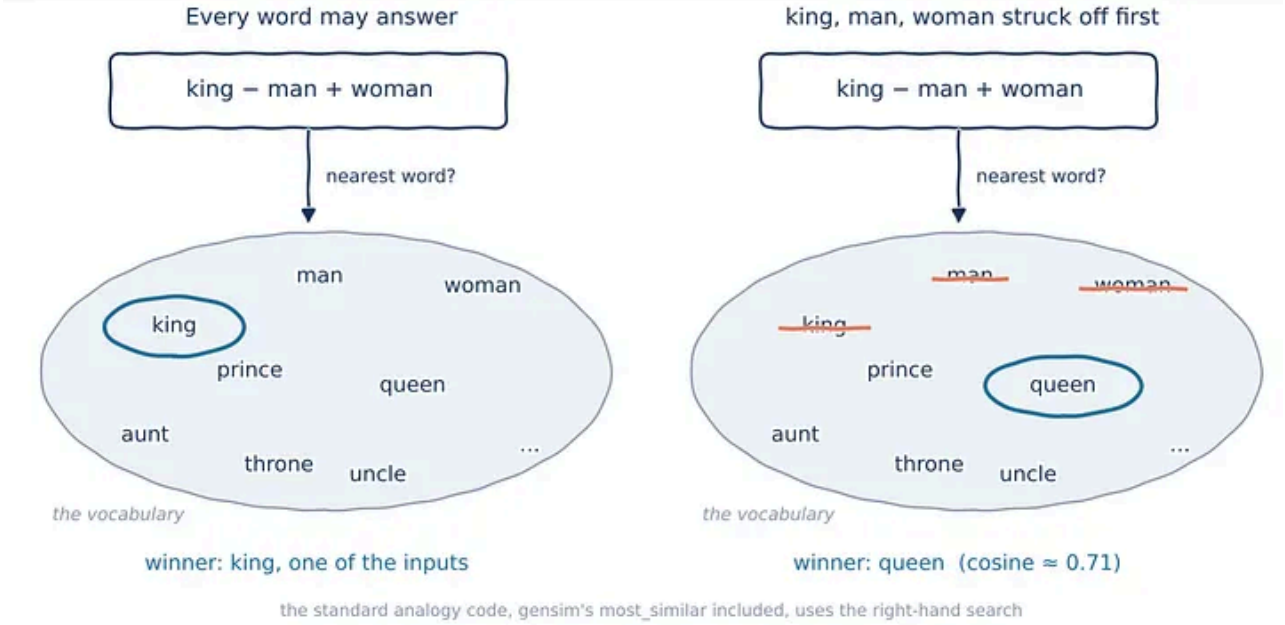
	crown	throne	he	she	the	of
king	33	20	66	6	165	110
queen	25	15	4	50	124	83
man	16	9	376	31	940	627
woman	14	8	28	339	846	564



A toy count table, re-read as PMI: subtract man's row from king's and woman's from queen's, and the two differences nearly match.

The GloVe paper builds its design on the same observation. *Solid* turns up far more often near *ice* than near *steam*, *gas* the other way round, and *water*about equally near both, so the ratio singles out the contexts that tell two words apart. Later work, by Arora and colleagues in 2016 and more precisely by Ethayarajh, Duvenaud and Hirst in 2019, pinned the condition down for an idealized embedding. The analogy requires that *king* keep company with *man* to the same degree that *queen* keeps company with *woman*, so that a single co-occurrence statistic, computed once for each pair, comes out the same.

On the second question, whether the arithmetic really said *queen*, the standard analogy code, including the `most_similar` function in gensim that most people use to reproduce the demonstration, removes the three input words from the list of candidates before it looks for the nearest vector. Without that step, the vector closest to **king** − **man** + **woman** is **king**. The difference between *man* and *woman* is small compared with everything that makes *king* distinctive, so the shifted point stays nearer its starting word than to anything else. Tal Linzen raised the issue in a 2016 workshop paper on evaluating word analogies. In 2020 Malvina Nissim, Rik van Noord and Rob van der Goot made it the centre of a paper in *Computational Linguistics* titled "Fair Is Better than Sensational: Man Is to Doctor as Woman Is to Doctor". Their cleanest example uses the pronouns. With the three input words removed, the query *he* is to *doctor* as *she* is to ? returns *nurse*, an answer widely read as evidence that the vectors had absorbed gender stereotypes from their text. With the input words left in the running, the arithmetic returns *doctor*.



The same search twice: let the input words compete and king wins; strike them off, as the standard code does, and queen wins.

A commercial embedding service might return 1,536 numbers for a sentence, and a vector database ranks stored documents by the cosine between those lists, the ruler from the first section. The models that produce these vectors are far larger than word2vec and read whole passages instead of single words, but many of them are trained by a close relative of negative sampling, pulling genuine pairs together and pushing random pairs apart. How much of the PMI picture carries over to them is still being worked out, and it bears on what their geometry can be trusted to say. Probes of those larger clouds of points still draw an arrow and look for the nearest vector to where it ends, and each such probe has to decide, as the word2vec benchmark did, whether the starting words may count as answers.

Word Embeddings + | Word2vec + | Machine Learning + | Linear Algebra +

Artificial Intelligence +



Written by Mathcube

4.95K followers · 322 following

Following

Blogging about math, physics, and programming.